Sleep Analysis Challenge

Victor, Sourya, and Lauren

Project Problems and Goals

Our goal was to classify a person's stage of sleep (Arousal, NREM1, NREM2, NREM3, REM) using 7 1-minute signals, sampled at 200Hz:

O2-M1	posterior brain activity
E1-M2	left eye activity
Chin1-Chin2	chin movement
ABD	abdominal movement
CHEST	chest movement
AIRFLOW	respiratory airflow
ECG	cardiac activity

This classification was done in order to improve the detection, diagnosis, and treatment of non sleep apnea related sleep disturbances and improve overall sleeping quality.

Project Steps

Loading the Dataset

Loading Problem in Memory

Processing Labels

Shuffling + Partitioning

Implementing the model

Adjusting model parameters

Testing the model

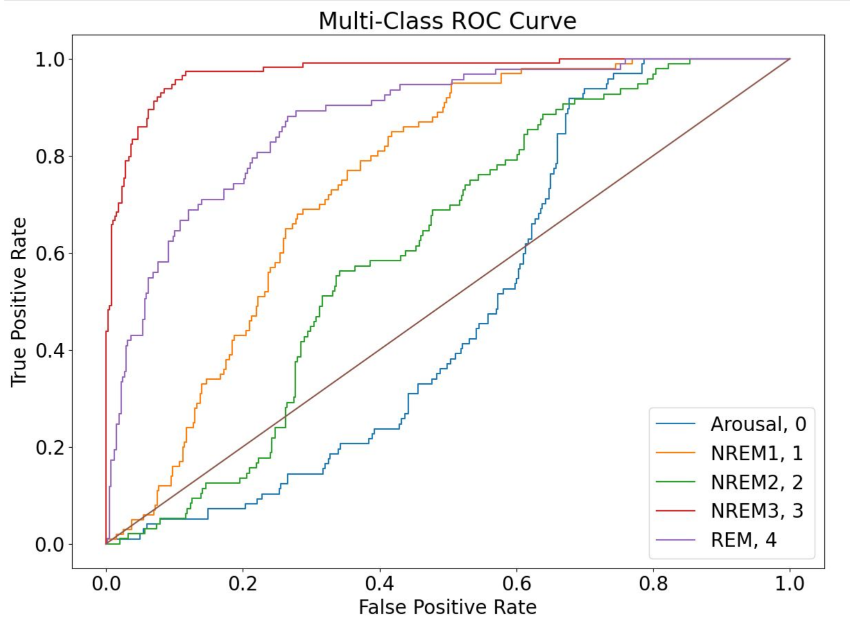
Evaluating the model

Model 1

0.768 ROC
Batch Size = 400

- Utilizes a lot of parameters
- Uses 4 convolutional layers, each followed by 1 ReLU layer and pooling layers, going up to a size of 128
- Implements constant dropout layers; however, this had led to much slower learning from the model, making it converge very early

```
nn.Conv1d(7, 16, 5, padding = 2),
  nn.ReLU(),
  nn.MaxPool1d(2),
  nn.Dropout(0.1),

  nn.Conv1d(16, 32, 3),
  nn.ReLU(),
  nn.MaxPool1d(2),

  nn.Conv1d(32, 64, 3, padding = 2),
  # nn.BatchNorm1d(64),
  nn.ReLU(),
  nn.MaxPool1d(2),
  # nn.Dropout(0.4),

  nn.Conv1d(64, 128, 3),
  nn.ReLU(),
  nn.MaxPool1d(2),
  nn.Dropout(0.2),

  nn.AdaptiveAvgPool1d(1),
  nn.Flatten(),

  nn.Linear(128,128),
  nn.Linear(128, 5),
```



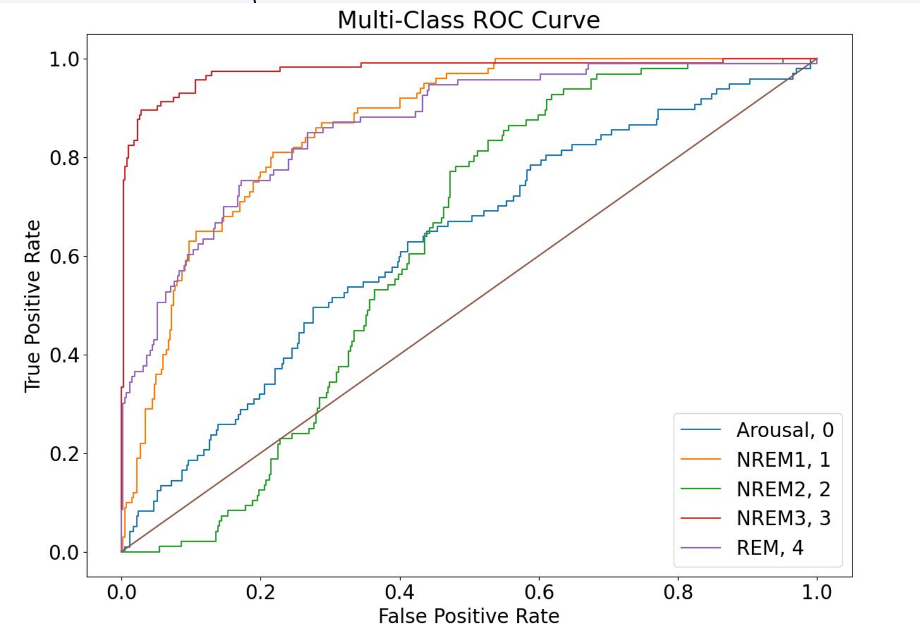

```
Epoch #59        Training Loss: 1.32    Training Accuracy: 0.39 Validation Loss: 1.22    Validation Accuracy: 0.49
Epoch #60        Training Loss: 1.09    Training Accuracy: 0.49 Validation Loss: 1.17    Validation Accuracy: 0.50
```

Model 2

0.814 ROC
30 epochs, Batch Size = 400

```
nn.Conv1d(7, 10, 9),
nn.ReLU(),
nn.MaxPool1d(10),
nn.ReLU(),
nn.Dropout(p=0.1),
nn.Conv1d(10, 16, 7),
nn.ReLU(),
nn.MaxPool1d(16),
nn.Conv1d(16, 32, 5),
nn.ReLU(),
nn.MaxPool1d(32),
nn.ReLU(),
nn.Flatten(),
nn.Linear(64,5)
```

- Less parameters
- Uses 3 convolutional layers, each followed by 1 ReLU layer and pooling layers
- Implements 1 dropout layer (p=0.1) after 1st convolution
- Achieved a best train accuracy of 0.95 and best validation accuracy of 0.67




```
Epoch #12  Training Loss: 0.49  Training Accuracy: 0.81 Validation Loss: 1.07 Validation Accuracy: 0.67
```

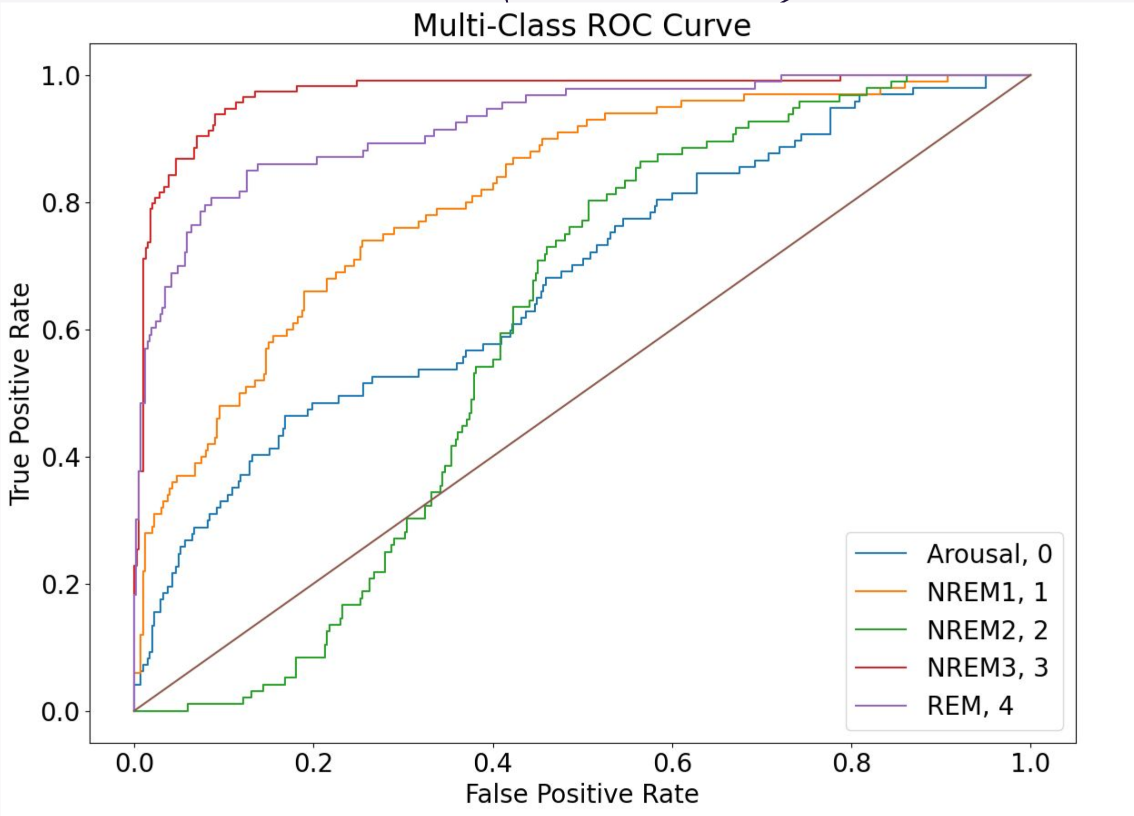
```
Epoch #29  Training Loss: 0.30  Training Accuracy: 0.92 Validation Loss: 1.25 Validation Accuracy: 0.66
Epoch #30  Training Loss: 0.29  Training Accuracy: 0.95 Validation Loss: 1.30 Validation Accuracy: 0.66
```

```
Epoch #59        Training Loss: 0.78    Training Accuracy: 0.76 Validation Loss: 1.03   Validation Accuracy: 0.59
Epoch #60        Training Loss: 0.95    Training Accuracy: 0.59 Validation Loss: 1.10   Validation Accuracy: 0.54
```

Model 3

0.821 ROC

```
nn.Conv1d(7, 32, 5),
nn.BatchNorm1d(32),
nn.LeakyReLU(),
nn.MaxPool1d(2),
nn.Dropout(0.3),
nn.Conv1d(32, 64, 5),
nn.BatchNorm1d(64),
nn.ReLU(),
nn.MaxPool1d(2),
   nn.Conv1d(64, 128, 5),
   nn.BatchNorm1d(128),
   nn.ReLU(),
   nn.MaxPool1d(2),
   nn.Dropout(0.4),
   nn.AdaptiveAvgPool1d(1),
   nn.Flatten(),
   nn.Linear(128, 64),
   nn.ReLU(),
   nn.Linear(64, 5)
```



Challenges

- Validation accuracy would plateau at 0.6, even as training accuracy would improve from 0.7 → 0.9
  - Means high overfit → add dropout layers
  - Tested dropout rates p = 0.1, 0.2, 0.4
  - Tested dropout layer placement
    - Work better after ReLU layers
  - Best results found when using dropout after 1 convolution + ReLU layer
    - This finding would change when we changed our model architecture
    - For our larger model, adding dropout layers did not seem to increase validation accuracy significantly

Challenges

- Validation accuracy would converge early
  - Need greater precision → used decaying learning rate
  - Tested decay rate of 0.8, 0.9
    - 0.8 seemed to work best
- Noisy data
  - Applied smoothing
  - Tested sliding windows of size 3, 5
    - Did not seem to improve model performance significantly

Future Steps

Different models

Experimenting with different neural network models like LSTMs or transformers could help to increase accuracy and lower loss

Experimenting with Pre-processing more

We mostly changed aspects about training the model, but changing how we clean/shuffle the data may have an impact on accuracy

Using more relevant data

Doing some more background research and finding more relevant data would help improve the effectiveness of our model

Thank You!